

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

Annexure A — Reverse Engineering

Project Analyzed: AI-Based Crop Recommendation System

Question 1 — System Decomposition & Analysis

Analyze an existing AI-based system and reverse engineer its architecture, identifying key components, data flow, and underlying algorithms.

The AI-Based Crop Recommendation System can be decomposed into three functional layers, each corresponding to a stage in the machine-learning lifecycle:

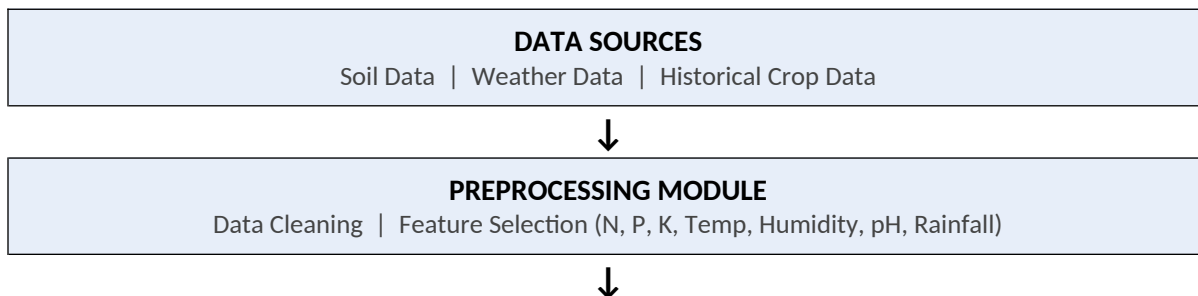
- **Data Layer:** Collects raw agricultural data — soil parameters (Nitrogen, Phosphorus, Potassium, pH) and climatic parameters (temperature, humidity, rainfall) — from historical records and agricultural datasets.
- **Processing / Intelligence Layer:** Cleans and formats the raw data, then trains and evaluates machine learning models (Random Forest, Decision Tree, KNN, SVM) to learn the relationship between soil-climate conditions and crop suitability.
- **Presentation Layer:** A user interface through which a farmer enters field data and receives the predicted crop along with a confidence/suitability score.

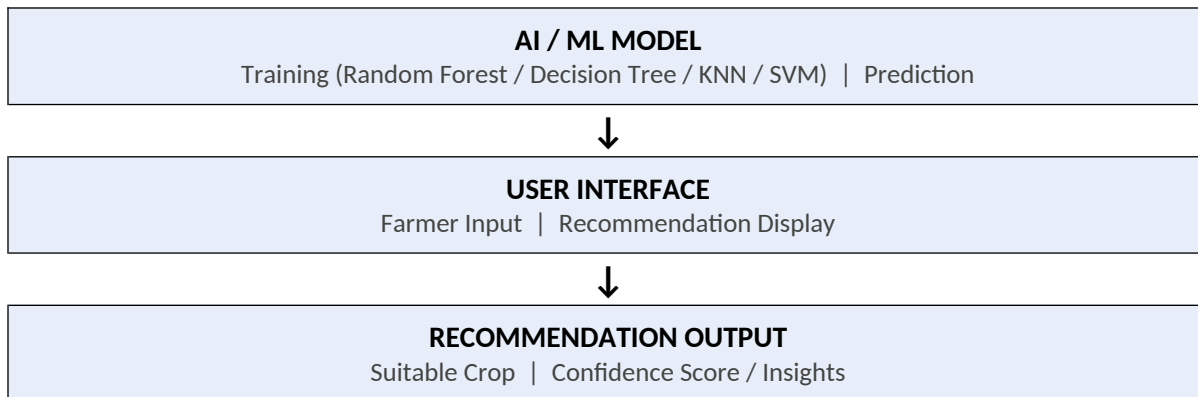
Reverse engineering the system's key components yields three modules: (1) Data Collection & Preprocessing, (2) AI Model Training & Crop Recommendation, and (3) User Interface & Recommendation Report. The data flow moves strictly in one direction — from raw data acquisition, through cleaning and feature preparation, into model training/inference, and finally to the output interface — with no feedback loop back into the dataset unless the system is manually retrained. The underlying algorithm family is supervised classification: among the candidate algorithms, Random Forest is highlighted as the primary technique because it is an ensemble method that combines multiple decision trees, handles multiple input features (the seven soil/climate parameters) well, and produces a reliable, low-variance crop prediction. Overall, the reverse-engineered architecture shows a linear, pipeline-style AI system rather than a distributed or multi-agent one — an important observation that motivates the improvements discussed in Question 5.

Question 2 — Architecture Reconstruction

Given a black-box software system, reconstruct its system architecture and explain the interaction between modules using appropriate diagrams.

Treating the crop recommendation tool as a black box and observing its inputs (soil and weather values) and outputs (a recommended crop), the following five-stage architecture can be reconstructed:





Module interaction is summarized below:

Module	Responsibility	Interacts With
Module 1 – Data Collection & Preprocessing	Gathers soil (N, P, K, pH) and climatic (temperature, humidity, rainfall) data; cleans and formats it; splits into train/test sets.	Sends processed dataset to Module 2 (AI Model Training)
Module 2 – AI Model Training & Recommendation	Trains ML models (Random Forest, Decision Tree, KNN, SVM) on the processed data, evaluates them, and selects the best-performing model to predict a suitable crop.	Receives data from Module 1; sends prediction results to Module 3
Module 3 – User Interface & Recommendation Report	Collects farmer input, forwards it to the trained model, and displays the recommended crop with a confidence score / suitability report.	Sends input to Module 2; presents output to the end user

Data flows strictly top-down through this pipeline. The Data Sources layer feeds the Preprocessing layer, which cleans and structures the data before passing it to the AI/ML Model layer for training and inference. The User Interface layer sits at the boundary between the human user and the model — it sends farmer-entered field data into the model and receives the predicted output, which is then rendered on the Recommendation Output layer as the suitable crop with supporting insights. Because each module has a single, well-defined responsibility and a one-directional interface to its neighbor, the reconstructed architecture follows a classic layered (pipe-and-filter) pattern common in applied ML systems.

Question 3 — Algorithm Identification

Reverse engineer a machine learning application to identify the algorithms, data processing steps, and decision-making logic used in the system.

Algorithms Identified: The system is built around supervised classification algorithms — Random Forest, Decision Tree, K-Nearest Neighbors (KNN), and Support Vector Machine (SVM) — trained on labeled historical crop data and compared to select the best performer. Random Forest is emphasized as the primary algorithm because, as an ensemble of decision trees, it aggregates multiple trees' predictions (reducing overfitting), naturally handles the seven numeric input features (N, P, K, temperature, humidity, pH, rainfall), and is well suited to multi-class crop classification.

Data Processing Steps identified through reverse engineering:

- 1. Data Collection — gathering agricultural datasets and historical records.
- 2. Data Cleaning — removing noise, missing values, and inconsistent entries.
- 3. Feature Selection — retaining the relevant soil (N, P, K, pH) and climate (temperature, humidity, rainfall) features.
- 4. Data Formatting — converting the cleaned data into a structure suitable for model training (e.g., numeric feature vectors).
- 5. Train/Test Split — partitioning the dataset so model performance can be validated on unseen data.

Decision-Making Logic: During training, each candidate model learns a mapping from the 7-dimensional feature vector to a crop label. For Random Forest specifically, many decision trees are grown on bootstrapped subsets of the training data; each tree independently splits on feature thresholds (e.g., "pH < 6.5") to arrive at a predicted crop class, and the forest outputs the majority-vote class across all trees. The models are then evaluated using Accuracy, Precision, Recall, and F1-Score, and the model with the best validation performance is selected as the deployed decision engine. At inference time, a farmer's newly entered soil/climate values are passed through this selected model, which applies the same learned splits/decision boundaries to output the single most suitable crop along with a confidence indicator.

Question 4 — Data Flow & Process Mapping

Examine an existing distributed AI application and map its data flow, processing pipeline, and communication between components.

Mapping the crop recommendation system's data flow end-to-end:

- Input Stage: Two data streams enter the system — (a) bulk historical agricultural data used for training, and (b) live field data entered by a farmer at prediction time (N, P, K, temperature, humidity, pH, rainfall).
- Preprocessing Stage: The historical data stream is cleaned, missing/invalid entries are handled, and it is split into training and testing subsets; the live farmer-input stream is validated and formatted to match the same feature schema used in training.
- Model Stage: The training subset drives model fitting (Random Forest and alternatives); the testing subset is used purely for evaluation and model selection. Once trained, the chosen model becomes a static inference service.
- Inference Communication: The User Interface module packages the farmer's validated input and sends it to the trained model (a function call / API-style invocation in this application); the model returns a predicted crop label and a confidence score.
- Output Stage: The interface renders the prediction as a recommendation report containing the original input values, the predicted crop, and the suitability/confidence level.

Communication between components is currently synchronous and centralized — the UI calls the model directly and waits for a single response, rather than distributing the workload across multiple nodes or services. This is consistent with the system being reverse-engineered as a monolithic pipeline. For the system to genuinely qualify as a distributed AI application (as implied by the course outcome), the data flow map above indicates clear seams where distribution could be introduced: the preprocessing stage could run as an independent, horizontally-scalable service (e.g., using PySpark RDD-style transformations for cleaning and feature engineering on large agricultural datasets), and the model-inference stage could be exposed as its own scalable microservice, decoupling it from the user-interface component so multiple farmer requests can be processed concurrently.

Question 5 — Improvement & Redesign

Perform reverse engineering on an intelligent system and propose enhancements or redesign strategies to improve scalability, efficiency, and performance.

Having reconstructed the architecture (Q2) and traced the data flow (Q4), the following redesign strategies are proposed to move the system from a single-machine, static pipeline to a scalable, production-grade intelligent system:

Area	Current Limitation	Proposed Enhancement
Data Sources	Relies on static, historical datasets	Integrate real-time weather APIs and IoT soil sensors (moisture, NPK sensors) for live data feeds
Scalability	Single-model batch processing not built for large-scale/distributed use	Deploy the pipeline on a distributed framework (e.g., Spark/PySpark) so preprocessing and model inference scale across nodes
Model Performance	Uses a single best-performing model chosen once	Use ensemble/stacking of Random Forest, SVM, and KNN with periodic retraining on new data to improve accuracy over time
Personalization	Generalized recommendation without location context	Add location-based and soil-type-specific recommendations using GPS/region metadata
Accessibility	Assumed desktop/manual data entry	Build a mobile application with an intuitive UI for on-field, real-time use by farmers
Scope of Output	Predicts only the suitable crop	Extend to yield prediction, irrigation scheduling, crop disease prediction, and market price insights

Architecturally, the redesign converts the current linear pipeline into a set of loosely-coupled services: a Data Ingestion Service (real-time weather API + IoT sensor feed), a Distributed Preprocessing Service (Spark/PySpark RDD/DataFrame transformations for cleaning and feature engineering at scale), a Model-Serving Service (the trained ensemble exposed via a REST/gRPC endpoint, allowing horizontal scaling and independent redeployment when the model is retrained), and a Client Layer (mobile app / web UI) that only communicates with the Model-Serving Service through well-defined APIs. This decoupling directly improves scalability (each service scales independently with farmer/request load), efficiency (distributed preprocessing avoids reprocessing the full dataset on every request), and performance (a dedicated serving layer with a periodically retrained, ensembled model improves prediction accuracy and responsiveness over time).

— End of Answers —